Tree Ensembles: Random Forests & Boosted Trees

BSAD 8310: Business Forecasting — Lecture 5

Department of Economics
University of Nebraska at Omaha

August 24, 2026

Lecture Outline

From Single Trees to Ensembles

Averaging many imperfect models can outperform any single model.

The Wisdom of Crowds

A single expert can be wrong. But average many independent estimates and errors tend to cancel out.

Classic Example: Guessing the Ox's Weight

At a 1907 county fair, 800 people guessed the weight of an ox. No individual was very accurate. But the **average of all 800 guesses** was within 1 pound of the true weight.

The same principle applies to models:

- A single decision tree is unstable and noisy (high variance)
- Average many trees trained on different versions of the data and the noise cancels out
- The combined forecast is far more stable than any individual tree

Ensembles work because: independent errors tend to cancel. The more independent (uncorrelated) the individual predictions, the more variance is reduced by averaging.

Bagging: Bootstrap Aggregating

Train many trees on bootstrap samples; average their predictions.

Bagging: The Big Idea

Problem from Lecture 4: a single decision tree has very high variance — change a few training points and you get a completely different tree. **Bagging solution (Breiman 1996):**

1. Draw B **bootstrap samples** from the training data (sample with replacement, same size as original)
2. Fit one deep decision tree on each bootstrap sample
3. **Average** all B predictions:

$$\hat{y}_{\text{bag}} = \frac{1}{B} \sum_{b=1}^{B} \hat{f}^{(b)}(\mathbf{x})$$

Why does this work?

$$\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

where ρ = pairwise prediction correlation. As $B \to \infty$, variance $\to \rho\sigma^2$.

Key insight: more trees always helps (never hurts). The variance floor is $\rho\sigma^2$ — determined by how correlated the trees are, not by the number of trees.

Out-of-Bag (OOB) Error: Free Cross-Validation

Each bootstrap sample uses about **63%** of the training observations (some appear multiple times; the rest are left out). The **out-of-bag (OOB)** observations are those *not* in a given bootstrap sample.

OOB error estimate:

1. For each observation i, collect predictions from all trees where i was NOT in the bootstrap sample ($\approx 37\%$ of all trees)
2. Average those predictions $\rightarrow \hat{y}_i^{\text{OOB}}$
3. Compute RMSE: OOB RMSE $\approx$ 5-fold CV RMSE

OOB error is essentially **free cross-validation**. No need to set aside a separate validation set or run CV. Use it for tuning hyperparameters.

`RandomForestRegressor(oob_score=True)`

Practical Tip

With large datasets, OOB error is very close to 5-fold CV. With small datasets (< 1000 obs.), use CV — OOB may be noisy.

Random Forests: Decorrelating the Trees

Restrict each split to a random subset of features so trees are less alike.

The Problem Bagging Does Not Fully Solve

Recall: $\mathrm{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$
Even with many trees, the variance floor is $\rho\sigma^2$ — determined by how *correlated* the trees are.

> **The correlation problem:** if one feature strongly dominates (e.g., last year's sales always gives the best first split), then almost all trees use that feature at the root. The trees end up highly correlated (ρ stays large), and bagging provides little variance reduction.

Random Forest solution (Breiman 2001): At each split, randomly sample only $m < p$ features and find the best split *among those* m only.

> Random feature subsampling **forces diversity** among trees: the dominant feature cannot always be chosen, so trees take different paths $\Rightarrow$ lower ρ $\Rightarrow$ lower ensemble variance.

Random Forest Hyperparameters

Parameter	sklearn name	Default	Effect
Number of trees	`n_estimators`	100	More = lower variance; plateau after ∼500
Max features/split	`max_features`	`'sqrt'`	Lower = more decorrelated; tune via OOB
Max depth	`max_depth`	None	None = full tree; limit to reduce memory
Min samples/leaf	`min_samples_leaf`	1	Higher = smoother predictions
OOB score	`oob_score`	False	Set True for free CV error estimate

Recommended starting point:

- `n_estimators=500`, `max_features='sqrt'`
- `min_samples_leaf=5` for smoother predictions
- `oob_score=True` to monitor without separate CV

Set `random_state=42` for reproducibility. Random forests use randomness in both bootstrap sampling and feature selection.

Feature Importance and Interpretation

Which predictors matter most? RF gives a data-driven answer.

Random Forest Feature Importance (Molnar 2022)

Mean Decrease in Impurity (MDI)

For each feature j: average the total reduction in node impurity across all splits on feature j, weighted by the fraction of observations in each node. Normalize to sum to 1.

Example: retail sales RF

Feature	Importance
Lag 12 (same month last year)	0.38
Lag 1 (previous month)	0.24
Rolling mean (12m)	0.15
Lag 2	0.10
December dummy	0.08
Unemployment rate	0.05

RF importance aggregates across **hundreds of trees**. More stable than a single tree's importance.

MDI can be biased toward high-cardinality features. Use **permutation importance** (`permutation_importance`) for a less biased measure on the test set.

Random Forest in Python

sklearn Code Template

```
from sklearn.ensemble import
RandomForestRegressor
rf = RandomForestRegressor(
n_estimators=500,
max_features='sqrt',
min_samples_leaf=5,
oob_score=True,
random_state=42)
rf.fit(X_train, y_train)
print(rf.oob_score_)
importances = rf.feature_importances_
```

Typical performance on RSXFS:

Model	Test RMSE
Seasonal naïve	≈3,800
Single deep tree	≈2,900
Random Forest	≈1,800

RF reduces RMSE by ≈35–40% vs. a single tree on this dataset.

Gradient Boosting

Instead of averaging independent trees, fit each new tree to the errors of all previous trees.

Boosting vs. Bagging: The Core Difference

Bagging (Random Forest):

- Trees built **in parallel**, independently
- Each tree tries to predict y directly
- Average reduces variance
- Adding more trees does not reduce bias

Boosting:

- Trees built **sequentially**
- Each tree fits the *errors* of the previous ensemble
- Adding more trees reduces both bias AND variance (carefully)
- Risk: can overfit if trees are too deep or too many

Analogy: Correcting a Forecast Iteratively

Forecast 1: "Sales will be \$50,000." Actual: \$60,000. Error: +\$10,000.

Forecast 2: "The first model was short by \$10,000."
Combined prediction: \$50,000 + \$10,000 = **\$60,000**.

Each stage corrects the remaining error from all previous stages.

The Gradient Boosting Algorithm

Gradient Boosting (for squared-error loss)

Initialize $F_0(\mathbf{x}) = \bar{y}$. For $b = 1, \ldots, B$:

1. Compute residuals: $r_i^{(b)} = y_i - F_{b-1}(\mathbf{x}_i)$
2. Fit a shallow tree T_b to $\{(\mathbf{x}_i, r_i^{(b)})\}$
3. Update: $F_b = F_{b-1} + \eta T_b$

Final prediction: $\hat{y} = F_B(\mathbf{x})$

The learning rate η:

- Small η (0.01–0.1): tiny step per tree — requires more trees but more robust
- Large η (> 0.3): fast but risks overshooting
- **Rule:** smaller η + more trees + early stopping typically wins

Trees are **shallow** (depth 3–6). Each corrects a small fraction of the total error. The ensemble slowly converges.

Warning: always use **early stopping** — without it, boosting will overfit.

XGBoost

Three improvements over vanilla boosting: second-order optimization, explicit regularization, and column subsampling.

XGBoost: Three Advances (Chen and Guestrin 2016)

1. **Newton step (2nd-order optimization):** Uses the gradient *and* curvature (Hessian) of the loss function to compute more precise tree-fitting directions. Converges faster than first-order boosting.
2. **Explicit regularization:** Penalizes the number of leaves (γ) and the magnitude of leaf weights (λ) directly in the objective. Guards against overfitting without relying on early stopping alone.
3. **Column subsampling:** Like Random Forests, randomly samples features per tree and per split. Reduces correlation and improves generalization.

> XGBoost dominated Kaggle competitions from 2014–2018. It consistently outperforms vanilla gradient boosting on tabular data.

Other popular options:

- `LightGBM` — faster on large datasets
- `CatBoost` — handles categorical features natively

XGBoost: Key Hyperparameters

Parameter	XGB name	Typical	Effect
Number of trees	`n_estimators`	500–2000	More + small η = better (use early stopping)
Learning rate	`learning_rate`	0.01–0.1	Smaller = more robust; needs more trees
Tree depth	`max_depth`	3–6	Shallow preferred; depth >6 risks overfit
Row subsample	`subsample`	0.7–0.9	Fraction of training rows per tree
Col. subsample	`colsample_bytree`	0.7–0.9	Feature fraction per tree (like RF)
L2 leaf penalty	`reg_lambda`	1	Ridge on leaf weights; stabilizes

Practical starting point: `learning_rate=0.05`, `max_depth=4`, `n_estimators=1000` with early stopping on validation RMSE. Tune `max_depth` and `subsample` via `TimeSeriesSplit`.

XGBoost in Python

Code Template

```
model = xgb.XGBRegressor(
n_estimators=1000,
learning_rate=0.05,
max_depth=4,
subsample=0.8,
colsample_bytree=0.8,
early_stopping_rounds=50,
random_state=42)
model.fit(
X_train, y_train,
eval_set=[(X_val, y_val)],
verbose=False)
```

Cross-validation for time series:

- Use `TimeSeriesSplit` with the sklearn API
- Reserve last 20% as final test set
- `early_stopping_rounds=50`: stops if val. RMSE stalls for 50 rounds

Typical RSXFS result: XGBoost RMSE $\approx$ 1,500–1,700 (vs. RF $\approx$ 1,800).

Key Takeaways and Roadmap

Bagging reduces variance; boosting reduces bias. Together they cover most of what tabular forecasting needs.

Tree Methods: Which One When

Method	Training	Interpretable	Tuning	Typical accuracy
Decision Tree	Single tree	High	Low	Baseline
Random Forest	Parallel (B trees)	Medium	Low	Good
XGBoost	Sequential (B trees)	Low	Higher	Best

Random Forest strengths:

- ✓ Robust — hard to overfit badly
- ✓ Minimal tuning; good out-of-the-box
- ✓ OOB error gives free cross-validation

Shared limitation:

- × Neither can extrapolate beyond the training range — a hard constraint for trending series

Practical order: start with a Random Forest — it is hard to make catastrophically wrong. Move to XGBoost when you have the time to tune and need the last few points of accuracy.

Lecture 5: Key Takeaways

1. **Ensembles** work because independent errors cancel. Averaging many imperfect models beats any single model.
2. **Bagging** averages B trees fit to bootstrap samples; variance falls as B grows but floors at $\rho\sigma^2$. **Random Forests** subsample features at each split to cut ρ — the key gain over plain bagging.
3. **OOB error** is free cross-validation: each observation sits out $\approx$37% of trees, giving an unbiased generalization estimate.
4. **Boosting** inverts the idea — trees are built sequentially, each fitting the ensemble's remaining errors. Bagging attacks variance; boosting attacks bias. **XGBoost** adds a Newton step, penalties on leaf count and weights, and column subsampling.
5. Tune by cross-validation, always with `TimeSeriesSplit` — a random split leaks the future into the past — and pair boosting with **early stopping**.

Next: Regularization & Model Selection — choosing among many predictors by shrinking coefficients rather than counting them.

References I

- Breiman, Leo (1996). "Bagging Predictors". In: *Machine Learning* 24.2, pp. 123–140.
- — (2001). "Random Forests". In: *Machine Learning* 45.1, pp. 5–32.
- Chen, Tianqi and Carlos Guestrin (2016). "XGBoost: A Scalable Tree Boosting System". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.
- Molnar, Christoph (2022). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 2nd. Open access. URL: https://christophm.github.io/interpretable-ml-book/.